

16-bit, 3-Stage Pipelined ALU

Stage-wise Progress, Results & Future Stage 3

Purpose: Show the professor what is completed in Stage 1 and Stage 2, their outputs, and the planned Stage 3 implementation.

1. Overall Architecture

Stage 1	ID/EX	Stage 2	EX/WB	Stage 3
DECODE	Pipeline Register	EXECUTE	Pipeline Register	WRITEBACK
Opcode, Rs1, Rs2, Rd Read operands	Opcode, A, B, Rd	16-bit ALU Result + Z, C, V, N	Result, flags, Rd	Result → register Flags committed

2. Stage 1 — Decode (Completed)

- Receive opcode, source registers and destination register.
- Read the two 16-bit operands from the register file.
- Store opcode, operands, destination and valid information in the ID/EX pipeline register.
- **Stage 1 does not calculate the ALU result.** It prepares the instruction for Execute.

Stage 1 — Core Verilog

```
always @(posedge clk) begin
  if (reset) begin
    id_valid <= 1'b0;
    id_opcode <= 4'b0;
    id_rd <= 3'b0;
    id_A <= 16'b0;
    id_B <= 16'b0;
  end else begin
    id_valid <= valid_in;
    id_opcode <= opcode_in;
    id_rd <= rd_in;
    id_A <= registers[rs1_in];
    id_B <= registers[rs2_in];
  end
end
```

Stage 1 result example: ADD R1,R2,R3 with R2=10 and R3=20 → Opcode=ADD, A=10, B=20, Rd=R1, Valid=1. These values are stored in ID/EX.

3. Stage 2 — Execute (Basic Implementation)

- Receive the opcode and operands from ID/EX.
- Perform the selected 16-bit ALU operation.
- Generate the 16-bit result and Z, C, V, N flags.
- Store the result, flags and destination register in EX/WB.

Stage 2 — ALU Connection and Pipeline Capture

```
alu_16bit ALU (
  .A(id_A), .B(id_B), .opcode(id_opcode),
  .result(alu_result),
  .Z(alu_Z), .C(alu_C), .V(alu_V), .N(alu_N)
);

always @(posedge clk) begin
  ex_valid <= id_valid;
  ex_rd <= id_rd;
  ex_result <= alu_result;
  ex_Z <= alu_Z; ex_C <= alu_C;
  ex_V <= alu_V; ex_N <= alu_N;
end
```

Stage 2 result example: A=10, B=20, opcode=ADD → Result=30, Z=0, C=0, V=0, N=0. These values are stored in EX/WB.

4. Stage 1 + Stage 2 Pipeline Timing

Cycle	Instruction 1	Instruction 2	Instruction 3
1	Decode	—	—
2	Execute	Decode	—
3	Writeback	Execute	Decode

This demonstrates overlapping execution: different instructions can occupy different stages in the same clock cycle.

5. Stage 3 — Writeback (Future Implementation)

- Take the result, destination register and flags from EX/WB.
- Write the 16-bit result into the destination register.
- Commit Z, C, V and N flags.
- Make the committed result available to subsequent instructions.
- Validate the Writeback stage with testbench cases.

Planned Stage 3 — Core Logic

```
always @(posedge clk) begin
    valid_out <= ex_valid;
    rd_out <= ex_rd;
    result_out <= ex_result;
    Z_out <= ex_Z; C_out <= ex_C;
    V_out <= ex_V; N_out <= ex_N;

    if (ex_valid)
        registers[ex_rd] <= ex_result;
end
```

6. Future Work — RAW Data Hazard Handling

Example: ADD R1,R2,R3 followed by ADD R4,R1,R5. The second instruction needs R1 before the first instruction has completed Writeback. This is a **RAW (Read After Write) data hazard**.

Planned solution: use forwarding where possible and insert a pipeline stall when the required value is not safely available.

7. What to Show the Professor

- Show the Stage 1 Decode code and demonstrate the ID/EX values.
- Show the Stage 2 ALU code and demonstrate result + Z/C/V/N flags.
- Run a short instruction sequence and show clock-by-clock pipeline movement.
- Explain Stage 3 Writeback and show the planned register update.
- Explain the RAW hazard example and the planned forwarding/stalling solution.

8. Three-Week Completion Plan

Period	Target
Presentation	Stage 1 + Stage 2 demonstration and pipeline-register flow
Week 1	Complete Writeback, operations and flag validation
Week 2	RAW hazard detection + forwarding/stalling + test cases
Week 3	Visualization, edge cases, report and final demonstration

Viva summary: Stage 1 decodes and prepares the instruction; Stage 2 performs the 16-bit ALU operation and generates flags; Stage 3 will commit the result and flags, followed by hazard handling and complete verification.

Based on the uploaded project plan: 16-bit, 3-stage Decode → Execute → Writeback pipeline with pipeline registers, status flags and hazard handling.